

USER MANUAL

Asterism

& AI Agents Map

Your AI agents, local models, MCP servers and skills on one map

Asterism 0.5.0 · AI Agents Map 0.13.1

October 2026

About this manual

This manual describes one application that comes in two editions:

Edition	Asterism	AI Agents Map
What it is	A stand-alone GTK 4 / libadwaita app for any Linux desktop	A plugin for the Omarchy desktop, opened from a button in the bar
Install from	AUR, AppImage or source	The Omarchy Plugin Marketplace
Look	Follows the system's light/dark mode and accent colour	Omarchy theme colours, or GNOME's Adwaita look
Settings folder	`~/.config/asterism/`	`~/.config/omarchy/ai-agents-map/`

Everything else — the cards, the map, connecting by drawing, local LLMs, models, MCP servers and skills — is the same in both. Where something applies to only one edition, it is marked ASTERISM or OMARCHY.

Conventions

Add	A button, tab or menu item in the window.
Settings › **Look**	A path through the interface: open Settings, then **Look**.
`~/.agents/skills`	A file, folder or command. `~` is your home folder.
Ctrl+B	A keyboard shortcut.

> **NOTE** Notes add useful background.

> **TIP** Tips show a quicker way or a good habit.

> **CAUTION** Cautions point out something that is hard to undo.

Contents

1 **Introduction** 1
- 1.1 What it does 1
- 1.2 Key ideas 2
- 1.3 Nothing changes without a preview 2

2 **Installation** 3
- 2.1 Asterism 3
- 2.2 AI Agents Map on Omarchy 3
- 2.3 First start 3
- 2.4 Removing the app 4

3 **The window at a glance** 5
- 3.1 Choosing what to show 5
- 3.2 Working with the map 6
- 3.3 Health check 6
- 3.4 Hints and updates 6

4 **Agents** 7
- 4.1 Agent cards and details 7
- 4.2 Starting and stopping agents 7
- 4.3 Hiding agents and providers 7
- 4.4 Updating agents 8
- 4.5 Agents the app can install 8
- 4.6 New agents 8

5 **Local LLMs** 9
- 5.1 Installing a local LLM 9
- 5.2 Starting and stopping with agents 9
- 5.3 GPU memory and Free GPU memory 10
- 5.4 Used by other apps 10

6 **Models** 12
- 6.1 Get from Hugging Face 12
- 6.2 Installed 13
- 6.3 Context size 13
- 6.4 On this PC 13
- 6.5 Moving and deleting models 13
- 6.6 One models folder 13

7 **Connecting by drawing** 14
- 7.1 Agent ↔ local LLM 14
- 7.2 Agent ↔ MCP server 14
- 7.3 Agent ↔ skill folder 14
- 7.4 When a connection fails 14

8 **MCP servers** 15
- 8.1 Where servers come from 15
- 8.2 Setting a server up 15
- 8.3 Managing servers 15

9 **Skills** 17
- 9.1 Finding skills 17
- 9.2 Skill folders 17

10 **Settings and Help** 18
- 10.1 Settings 18
- 10.2 Help 18
- 10.3 Disk space 19

11 **Healthy & Security check** 20
- 11.1 Reading the card 20
- 11.2 Changing a setting 21
- 11.3 What is checked for each agent 21
- 11.4 Managing the risk 22
- 11.5 The words explained 22

12 **Privacy and safety** 24

13 **Troubleshooting** 25

A **Reference** 26
- A.1 Keyboard shortcuts 26
- A.2 Files and folders 26
- A.3 Glossary 27

CHAPTER 1

Introduction

Asterism shows the AI setup on your computer as cards and as a map: which coding agents are installed, which language models they use — cloud services and local runtimes — the models those serve, and which MCP servers and skills each agent has. And it lets you change that setup by drawing lines.

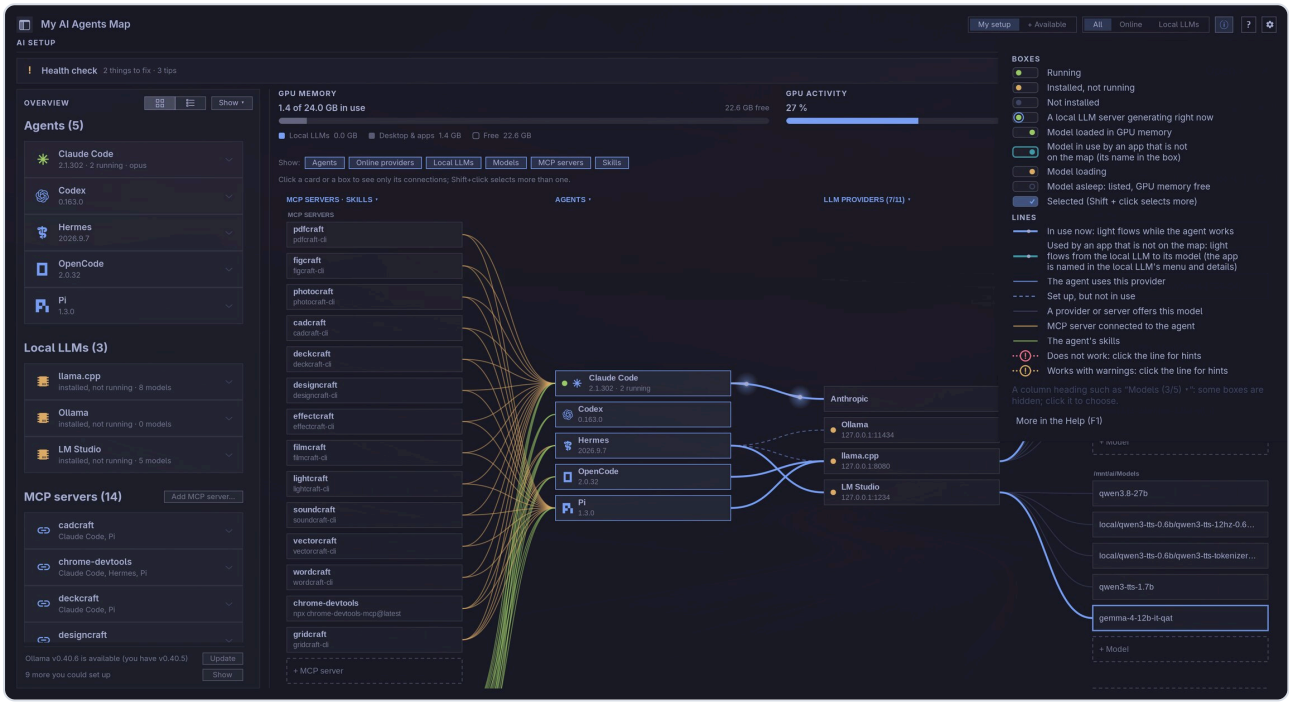


Figure 1 · The main window with the cards on the left, the map on the right and the legend open.

1.1 What it does

- **Coding agents** — Claude Code, Codex, GitHub Copilot, Cursor CLI, Gemini, OpenCode, Pi, Hermes and more: their version, plan, limits and usage, and what each one still needs, with the command that fixes it.
- **Local LLMs** — llama.cpp, Ollama, LM Studio, vLLM, Jan and KoboldCpp: install them, download models, set the context size that fits your graphics card, and start them together with the agents that use them.
- **MCP servers and skills** — found on your computer or on the web, with recommended servers and a search of the official MCP registry.
- **A health check** — what works, what to fix and what your setup is good for.

1.2 Key ideas

Agent	A program you talk to that writes code or does tasks, such as Claude Code.
LLM provider	Where an agent's model runs: a cloud service (Anthropic, OpenAI, …) or a local LLM runtime on this computer (llama.cpp, Ollama, …).
Model	The language model itself. Local models are files, usually in GGUF format.
MCP server	A small program or web service that gives agents extra tools — reading files, searching the web, controlling a browser. MCP is the Model Context Protocol.
Skill	A folder with instructions (a `SKILL.md`) and sometimes scripts that teach an agent how to do a particular job.

1.3 Nothing changes without a preview

Every change to an agent's settings is shown first, line by line, before anything is written. The old file is kept as a backup, and the bar that appears after the change has **Undo**. The app never asks for administrator rights.

CHAPTER 2

Installation

2.1 Asterism ASTERISM

Choose one of these ways:

- **Arch Linux (AUR):** install the package `asterism` with your AUR helper.
- **AppImage:** download `Asterism-<version>-x86_64.AppImage` from the releases page at `github.com/uBruckhaus/asterism`, make it executable and run it. The AppImage uses your distribution's Python 3 with PyGObject, GTK 4 and libadwaita:

Fedora	`sudo dnf install python3-gobject gtk4 libadwaita`
Ubuntu / Debian	`sudo apt install python3-gi gir1.2-gtk-4.0 gir1.2-adw-1`
Arch	`sudo pacman -S python-gobject gtk4 libadwaita`

- **From source:** run `bin/asterism` in a checkout of the repository.

Agent icons need a Nerd Font (Symbols Nerd Font). To run agents inside the window, also install VTE for GTK 4 (`vte4`).

2.2 AI Agents Map on Omarchy OMARCHY

Install the plugin from the Omarchy Plugin Marketplace, or from a terminal:

```
omarchy plugin add https://github.com/uBruckhaus/omarchy-ai-agents-map.git --enable
```

A robot button appears in the bar. Everything else it needs comes with a standard Omarchy install. To run agents inside the window, install VTE: `sudo pacman -S vte4`.

2.3 First start

On the first open, the app asks whether to add itself to the app launcher. Nothing is written there unless you say yes; you can change it later in Settings › **In the app launcher**.

The window is a normal app window: it can be tiled, floated, moved to another workspace and reached with `Alt`+`Tab`. OMARCHY The bar button opens the window, brings it to the front, or hides it when it is already in front. Closing the window hides it; the program keeps running in the background so it opens again at once.

> **NOTE** While an agent runs inside the window, closing it asks first: **Stop and close** (the agent's local LLM then stops too and frees the graphics card's memory) or **Keep running**.

2.4 Removing the app

OMARCHY `omarchy plugin remove ubruckhaus.ai-agents-map`. ASTERISM Remove the package or delete the AppImage.

Before removing, switch off Settings › **Start local LLMs with their agents** so the background watcher is removed too, and Settings › **In the app launcher** to delete the launcher entry. Changes you made with the app stay: agents' settings, MCP servers, skills and installed local LLMs. Uninstall a local LLM first (right-click › **Uninstall…**) if you want it gone. To delete the app's own state as well:

```
rm -rf ~/.config/omarchy/ai-agents-map ~/.config/asterism \
       ~/.local/state/ai-agents-map ~/.cache/ai-agents-map
```

CHAPTER 3

The window at a glance

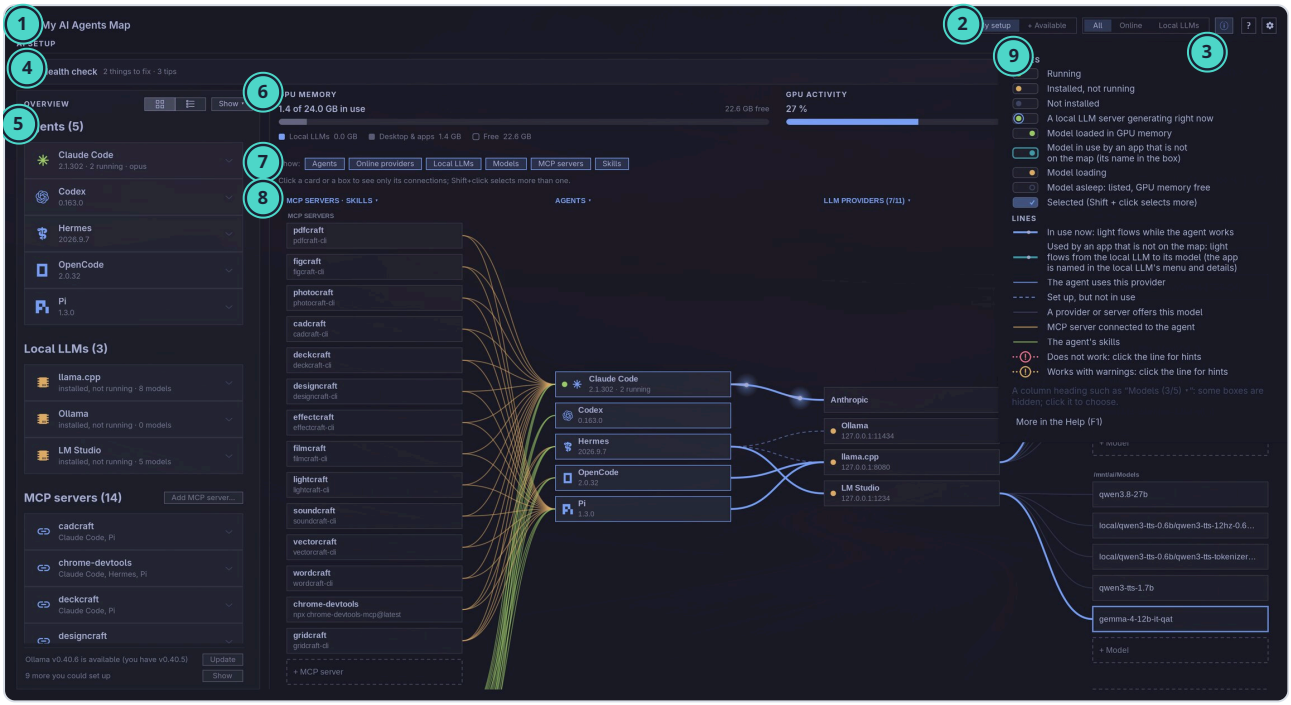


Figure 2 · The main window. The numbers match the list below.

1. **Sidebar button** — hides or shows the cards (`Ctrl`+`B`).
2. **My setup · + Available** and **All · Online · Local LLMs** — what to show (see below).
3. ⓘ legend, **?** Help (`F1`) and the gear for Settings (`Ctrl`+`,`).
4. **Health check** — a one-line summary; **Open ›** lists the details.
5. **Cards** — Agents, Local LLMs, MCP servers, Skills, each with its **Show ▾** list. The buttons beside **Show ▾** switch between cards and a compact tree.
6. **GPU memory** and **GPU activity** — how the graphics card is used right now.
7. **Show** toggles — shorten the map to the categories you are working on.
8. **The map** — MCP servers and skills ← agents → LLM providers → models.
9. **The legend** — what every dot, line and mark on the map means.

3.1 Choosing what to show

Switch	Shows
My setup	Only what is yours: installed agents and local LLMs, and MCP servers an agent uses. This is the default.
+ Available	Adds what you could set up: agents and local LLMs that are not installed, MCP servers no agent uses, and anything you have hidden.

Switch	Shows
All	Everything.
Online	Online providers only; no local LLMs, their models or the GPU meters.
Local LLMs	Only local LLMs and the agents that can use them.

Settings › **Start with** picks the view the app opens with. Every column heading on the map (**AGENTS** ▾, **MODELS** ▾, …) opens a list where you can untick single boxes or whole groups; the heading then shows how many are shown, for example *(5/8)*.

3.2 Working with the map

- **Hover** a box to light it up together with everything it is joined to.
- **Click** a box or a card to select it (✓); `Shift`+click adds more. `Esc` or **Clear selection** clears it.
- The **pin** at a box's top-right corner keeps its connections lit while you look elsewhere.
- **Right-click** a box for everything its card offers: start or stop, details, install, models, **Hide**, and more.
- Each agent's active path — agent → provider in use → its model — is drawn bold. While an agent is working, light runs along that line.
- When a program that is not on the map uses a local LLM, the line from that LLM to the model turns bold in a colour of its own (purple or cyan, whichever stands out from the theme's other colours), and light runs along it. See Used by other apps.
- Zoom with **− 100 % +**, `Ctrl`+scroll or `Ctrl`+`+` / `-` / `0`.
- Drag the line between the cards and the map to make the cards wider; double-click it to go back to the default width.

> **TIP** In a narrow window (below about 1300 pixels, such as half the screen) only the cards are shown and pages open over them. Maximize the window to see the map.

3.3 Health check

The **Health check** line above the cards sums up how well the setup works, for example *1 thing to fix · 3 tips*. **Open ›** lists by area — this PC, agents, local LLMs and models, tools — what is fine, what are tips and what to fix, each with what to do and, where possible, a button that does it. It also tells you what the setup is good for (everyday help, coding, images, voice, research and writing) and the next step. The health check only reads; it never changes, starts or sends anything.

3.4 Hints and updates

Below the cards, a hint area appears when there is something to say: new programs or skills found on this PC, updates for installed agents and local LLMs, and how many more things you could set up. Updates are checked in the background when the window opens, at most every six hours. Nothing is installed by the check itself; an update starts only when you click **Update**.

CHAPTER 4

Agents

4.1 Agent cards and details

Every coding agent is listed, installed ones first with their version; the default agent is marked. Click ⓘ **Details** on a card to see:

- whether the agent is ready (and if not, why), whether it is running now or when it was last used;
- today's and the last seven days' tokens and sessions, the plan, a chart of tokens per day, rate limits with reset times and tokens by model;
- **Healthy & Security check**: every setup check with a ✓, or what to do next — sign in with which command, which subscription or API key it needs, install or start its local LLM, download a model, add MCP servers or skills — and what the agent may do without asking you, with what lowers the risk (see Healthy & Security check);
- **Configuration**: the agent's settings, MCP servers, skills and settings folder.

Claude Code's details also show the last 30 days per project folder: working time, sessions, tokens, lines changed and what the tokens would cost at API prices.

ASTERISM Claude Code's details also list **Topics**, the titles Claude Code gave its sessions, and offer **Summarize with Claude…**. Before anything is sent, you see exactly what will be sent: the titles and numbers, never the conversations.

4.2 Starting and stopping agents

- **Start here** runs the agent in a terminal inside the window, beside the cards. This needs VTE (`vte4`).
- **Start in terminal** opens it in a terminal window of its own.
- While it runs inside the window: **Show terminal** and **Stop**. Stop ends the agent and everything it started, such as its MCP servers.

When agents have terminals in the window, a tab line above the map shows one tab per terminal with a dot for *working*, *running* or *ended*. Click a tab to show that terminal; **×** closes an ended one.

An agent with an open instance anywhere on the computer gets a green dot and *running*, and the model box it runs with says *in use · <agent>*.

4.3 Hiding agents and providers

Hide (on a card, in the tree, or in a box's right-click menu) leaves an agent or a provider out of **My setup**, with Undo. Hidden ones are listed under **+ Available › Hidden** with **Show again**. A provider can only be hidden while no shown agent uses one of its models.

4.4 Updating agents

When a newer version of an installed agent is out, the hint area names it. **Update** runs the update the same way the agent was installed — for example `mise upgrade <tool>` or `claude update` — in a terminal window of its own, so you can follow it.

4.5 Agents the app can install

Three agents can be installed, updated and uninstalled from the map (right-click the agent › **Install…**):

- **DeepSeek Harness** (`dsh` , developer preview), from npm. It can use local LLMs.
- **Antigravity CLI** (`agy` , Google), from Google's release manifest; the download is checked against its SHA-512.
- **LM Studio Bionic** cannot be installed from here (no published checksum); get it from lmstudio.ai. It always uses LM Studio's engine and models.

4.6 New agents

When an agent turns up that the app does not know yet, it asks an installed agent (Claude Code, else Codex) once to find out on the web where that agent keeps its skills and MCP servers. The answer is checked before use, and nothing in it is ever run. Settings › **Look up new agents** switches this off.

CHAPTER 5

Local LLMs

A local LLM runtime runs language models on your own graphics card, so prompts never leave the computer. The app installs, starts and stops them for you.

5.1 Installing a local LLM

Right-click a grey (not installed) local LLM on the map › **Install…**. You choose the program folder, the model folder (the one your other local LLMs use is suggested) and whether it starts at login. It is installed for your user only, with a user service, so no password is needed. Every download must match the SHA-256 checksum its release publishes before it is unpacked or run.

Runtime	Notes
Ollama	The official release (with ROCm for AMD cards).
llama.cpp	The build of your choice (Vulkan, ROCm, CUDA, CPU), run as a model router: every GGUF in the model folder, one loaded at a time.
LM Studio	Not installed from here (no published checksum); get it from lmstudio.ai. Shown and managed once installed.
vLLM	For NVIDIA cards, in its own Python environment.
Jan	Its AppImage, unpacked, with Jan's engine as a background service for agents.
KoboldCpp	The official single-file build, run as a model router.

Installed this way, a local LLM also offers **Update…** (the previous version is kept as a fallback), **Model folder…** and **Uninstall…**, which removes the program, command, service and settings but never the models. Local LLMs you installed yourself are shown, but never changed.

5.2 Starting and stopping with agents

Connect an agent to a local LLM once, then just start the agent — even with the app closed or after a reboot. Its local LLM starts with it, and when the last agent using it closes, the LLM stops about five seconds later, so its graphics memory is free again.

A small background service, `ai-agents-map-providers.service`, does this. It is set up with the first connection to a local LLM and removed when the last one goes, or when you switch off Settings › **Start local LLMs with their agents**.

> **NOTE** KoboldCpp takes a few seconds to start, longer than some agents wait. So it starts at login, and when the last agent closes only its model is unloaded.

5.3 GPU memory and Free GPU memory

Above the map, **GPU memory** shows how much is in use, split into local LLMs, desktop and apps, and free; **GPU activity** shows load, temperature and power. Each local LLM's box shows how much memory it holds, and loaded models get a green dot.

Right-click a local LLM for **Start**, **Stop**, start at login, and **Free GPU memory**: its models are unloaded, it keeps running and loads one again when asked.

5.4 Used by other apps

Local LLMs are not only used by agents. A plugin such as Video Trans, a chat app or your own script can send requests to them too. The app notices this and shows it:



Figure 3 · A program that is not on the map (here `curl`) generating with llama.cpp: the cyan line and box (Tokyo Night theme). Claude Code's own path to opus stays in the accent colour.

- **On the map**, the line from the local LLM to the model it is generating with turns bold in its own colour, light runs along it, and the model box gets a frame and dot in that colour and says who is using it, for example *in use · Video Trans*.
- **The local LLM's right-click menu** and its **Details** page say *In use by Video Trans · now*, or how many seconds ago it was last used.
- The legend shows both marks.

The app looks at which of your own programs have a connection open to the local LLM. It never reads what they send. A plugin is named by its own name, an agent by its name, anything else by its program name. The app itself and LLM Monitor, which only read the status, are not counted.

> **NOTE** The light shows only while the program is generating, and fades about ten seconds after its last request. A program that is open but idle — Video Trans with no speech to translate — shows nothing. A program stays listed in the menu for half a minute after its last request.

CHAPTER 6

Models

Right-click an installed local LLM › **Models…**, or click a folder's dashed **+ Model** box on the map. The page has a list on the left and the chosen model on the right, in three tabs.

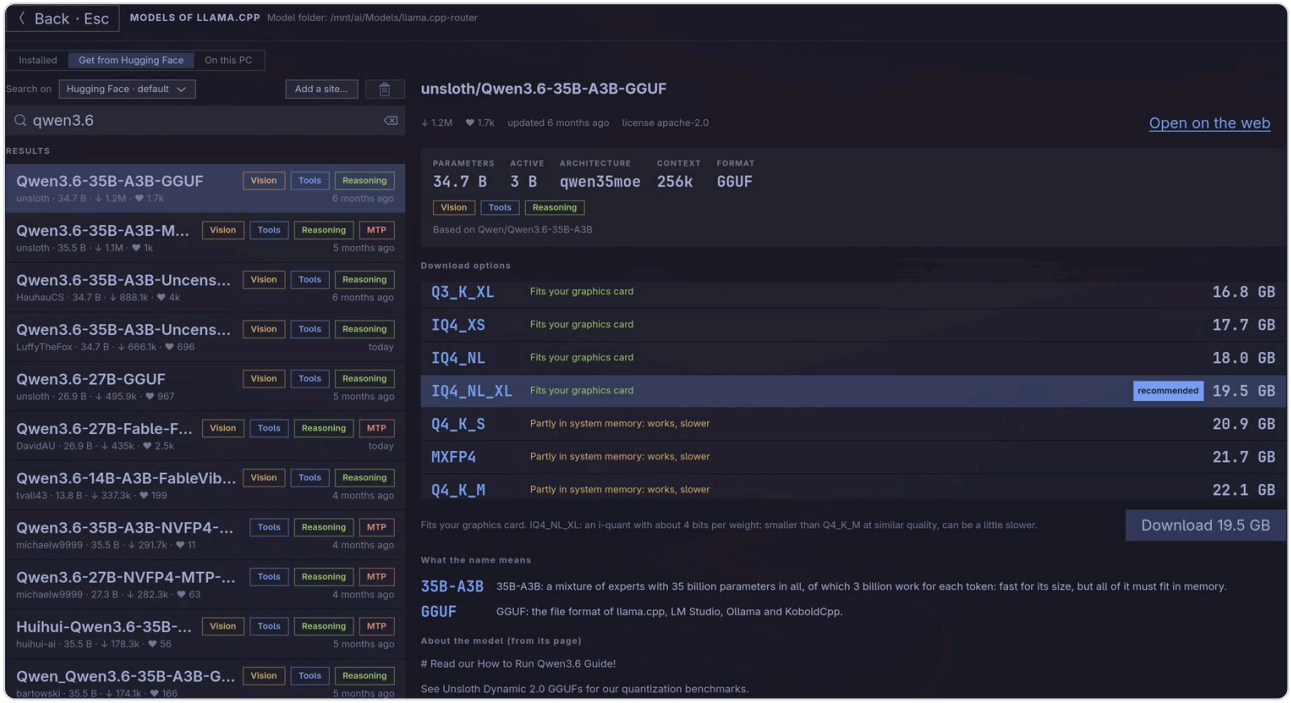


Figure 4 · Get from Hugging Face: what a model can do, and which size fits your graphics card.

6.1 Get from Hugging Face

Browse the trending GGUF chat models or search. Each result shows its author, size, downloads, likes, age and what it can do: **Vision**, **Tools**, **Reasoning**, **MTP**. The model's page shows its parameters, architecture, context, license and model card. **What the name means** explains the parts of the name, such as *35B-A3B* or *Q4_K_M*.

Download options lists every size with how it fits: in the graphics card, partly in system memory (works, but slower), or likely too large. The best fit is chosen for you; **Download** gets it into the model folder.

Downloads run in the background and show in a strip at the top of the main page. A broken connection is retried and the download resumes. If the app was closed during a download, it is listed as interrupted, with **Continue** or **Discard**.

6.2 Installed

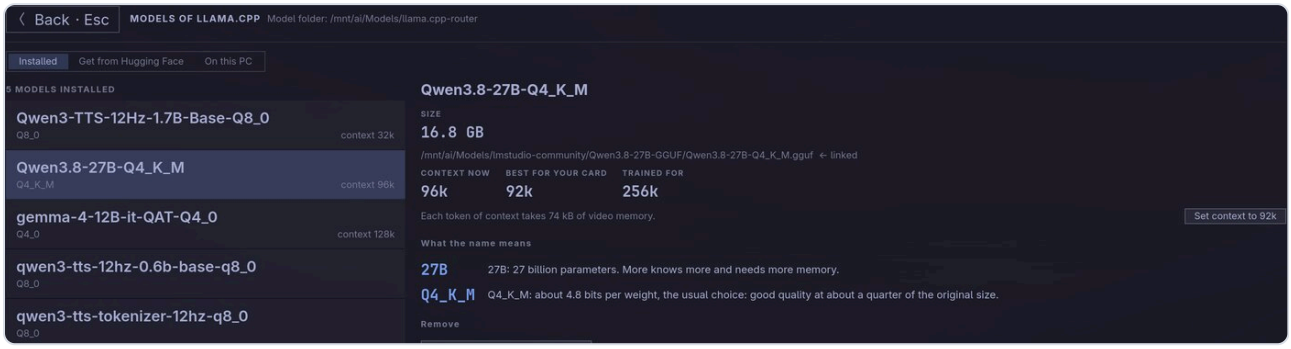


Figure 5 · Installed: context now, best for your card and trained for; what the name means; Remove.

Each model shows its size, file and context — *now*, *best for your card* and *trained for*. **Remove** asks first and moves the files to the trash, where you can restore them. A model that is only a link loses only the link.

6.3 Context size

The context is how much text the model can keep in mind at once. The app calculates the largest context that fits your graphics card from the model file itself. **Set context** writes it to the local LLM's settings, with a backup and Undo. Agents need a fairly large context; when you connect an agent to an LM Studio model without a context set, the best one is set in the same step.

6.4 On this PC

Lists GGUF files the local LLM does not read yet — in its models folder, the other local LLMs' folders, `~/Downloads` and `~/Models`. **Add** makes one its model without a second copy (a link), except for Ollama, which imports its own copy.

6.5 Moving and deleting models

Right-click a model of llama.cpp, KoboldCpp or LM Studio › **Move to…**, or drag the model box onto another local LLM's folder. First you see how much moves and which agent uses the model. Undo moves the files back. **Delete…** asks first and moves the files to the trash.

6.6 One models folder

Settings › **Models folder** sets one folder for every local LLM's models. **Reorganize models…** previews how the files would be sorted (`<folder>/<publisher>/<model>/`), and nothing changes until you press **Reorganize**. With **Keep old paths working** (on by default), a link stays at every old place. **Undo the last reorganize** puts everything back.

> **CAUTION** Moving models to another disk copies them first and removes the originals only when the copy is complete. Make sure the target disk has enough free space; the app refuses the move if it has not.

CHAPTER 7

Connecting by drawing

Press on a box on the map and drag to another to connect them. Every change shows exactly which lines of which file change before anything is written, keeps a backup, and can be undone from the bar that follows.

7.1 Agent ↔ local LLM

Drag from an agent to a local LLM, or back. Choose the model and whether the agent uses it right away. To change a connection, click the line to select it, then drag its end to another local LLM — or onto empty space to remove it. With an agent selected, clicking a model of a connected local LLM makes that model the agent's model.

Supported agents: Hermes, Pi, OpenCode and DeepSeek Harness.

7.2 Agent ↔ MCP server

Drag from an agent to an MCP server. The server's settings are copied into that agent's own format. Supported: Claude Code, Codex, Copilot, Crush, Cursor, Gemini, Grok, Hermes, Muse, Oh My Pi, OpenClaw, OpenCode and Pi. Settings files that may hold keys are always written so that only you can read them.

7.3 Agent ↔ skill folder

Drag from an agent to a skill folder. Pi and Hermes get the folder in their settings; every other agent gets one link per skill in its own skills folder. A folder an agent reads by itself (most read `~/.agents/skills`) cannot be taken away from just that agent.

7.4 When a connection fails

Right after a connection is written, the app checks whether it will work: whether a local LLM is installed and offers the model, whether an MCP server answers, whether the skill links are intact. A connection that fails stays on the map as a **dotted red line** with a **!**; one with only warnings is **dotted yellow**. Click the line or its **!** for hints:

- **Test again** checks once more; when it passes, the line becomes normal.
- **Try again…** opens the connect page for a connection that was never written.
- **Remove connection** removes it.

CHAPTER 8

MCP servers

MCP servers give agents extra tools. **+ MCP server** on the map or under the cards opens the page for adding one: a list on the left, the chosen server's page on the right.

8.1 Where servers come from

Tab	Contents
Recommended	The MCP project's reference servers (Filesystem, Fetch, Git, Memory, Sequential Thinking, Time, Everything) and official servers of well-known makers (GitHub, Playwright, Chrome DevTools, Brave Search, Context7), grouped by purpose and pinned to their newest version.
MCP registry	A search of the official MCP registry. Each result names its publisher as the registry checked it. The search can take ten seconds or more.
On this PC	Programs on this computer that offer an MCP server, found by reading their files — nothing is started. AppImages are only looked into when you click **Look inside…**.
By hand	A command line, or a web address with headers.

8.2 Setting a server up

The server's page says who made it and how well that is checked, what it does (with things to ask an agent once it has it), how it runs — a program through `npx`, `uvx` or Docker, or a web service — and what it needs, such as a folder or an API key with a link to where you get one. Below it, **Set it up**:

1. Fill in the fields the server needs.
2. Tick the agents that should get it.
3. Click **Test** to ask the server for its tools.
4. Click **Add**: all ticked agents are written at once, with one preview and one Undo.

> **NOTE** Secret values such as API keys are passed in the environment or in headers, never on a command line where other users of the computer could read them. A server that would need a secret on its command line has to be set up by hand.

8.3 Managing servers

Details on an MCP server card shows each agent's settings file, the command or address, and on request the server's tools; values of keys are never shown. **Remove** takes the server from every agent. Servers stay in the list when no agent uses them any more, so a line can connect them again.

When a newer version of an app that provides an MCP server turns up in `~/Applications`, a note names it. **Update…** unpacks the new version, points every agent at it and keeps the old one as a backup for seven days.

CHAPTER 9

Skills

A skill is a folder with instructions that teach an agent a particular job. **+ Skills** opens a page laid out like the MCP page.

9.1 Finding skills

- **On this PC**: skill folders that apps brought along and no agent reads yet.
- **From the web**: a search of the skills.sh directory, another site with the same API, or a GitHub repository with skills. A skill is fetched from its GitHub repository into a folder of your choice, by default `~/.agents/skills`.

> **CAUTION** Skills can contain scripts that an agent may run later. The page shows the repository and lists the scripts before anything is fetched — check them.

9.2 Skill folders

Each skill folder card shows its skills and the agents that read it. A folder that is shared through links shows up once, where it really lives. **Details** lists every skill with its description and how each agent gets the folder.

CHAPTER 10

Settings and Help

10.1 Settings

The gear (`Ctrl`+`,`) opens the settings. `Backspace` or `Esc` goes back.

Setting	What it does
Language	The system language, or English, Deutsch, Français, Español, Italiano, Português, Nederlands, Polski.
Look OMARCHY	The Omarchy theme colours, or GNOME's Adwaita look with the system's light/dark mode and accent colour.
Start with	Open with **My setup** or **+ Available**.
Sidebar	Cards or a compact tree.
Animate active lines	Light runs along the line of a working agent.
Terminal tabs above the map	One tab per agent terminal in the window.
Corners	Like windows, rounded or square.
Look up new agents	Ask an installed agent where an unknown agent keeps its settings.
Show what you could set up	The "N more you could set up" hint below the cards.
Start local LLMs with their agents	The background watcher described in Starting and stopping with agents.
Too narrow for the map OMARCHY	What "Maximize to see the map" does: move to a free workspace, maximize in place, or show no hint.
In the app launcher	The launcher entry; off removes it.
Agents start in	The folder agents start in.
Models folder	One folder for all models, with **Reorganize models…**.

10.2 Help

? (`F1`) opens Help: **Open user manual** opens this manual (the copy that came with your version) in your PDF viewer; **Disk space** (below); an animation of setting up by drawing; and LLM Monitor, a companion bar plugin that shows what a local LLM is doing (a small animation of its bar item and panel). Both animations, and the one in Settings › **Too narrow for the map**, are drawn in your theme's colours and follow a theme change at once. The ⓘ legend in the header explains every dot, line and mark of the map.

10.3 Disk space

Disk space: agents, local LLMs and models shows how much room your AI setup takes and where it is. It is measured in the background each time Help opens (again after ten minutes, or with **Measure again**), and only read — nothing is deleted from here. Its heading always shows the totals in one line (*Agents · Local LLMs · Models · In all*); click it to fold the section away. **Disks**, **Agents**, **Local LLMs** and **Models folders** inside start folded, each showing its sum and largest entries; click one to open it.

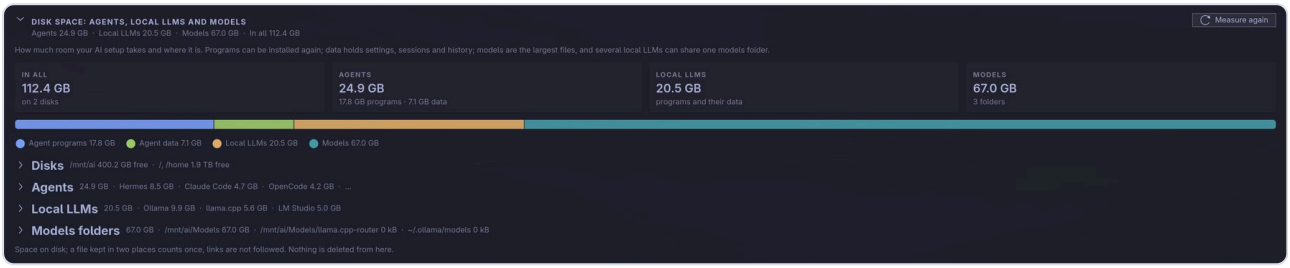


Figure 6 · The totals, the split, and the folded sections with their sums.

- **Tiles and bar**: everything together, then agent programs, agent data (settings, sessions, history), local LLMs (programs and their own data) and models. Hover over the bar for exact sizes.
- **Disks**: each disk with what the AI setup takes on it and how much is free.
- **Agents**, **Local LLMs**, **Models folders**: one row each, largest first. Open a row to see every folder with its size and number of files, and open it in the file manager. A program folder says how many versions are kept; for an agent with several old versions in mise, a tip names `mise prune`, which removes the unused ones. A models folder says which local LLMs use it, or that it holds only links to models in another folder.

> **TIP** Programs can be installed again and models downloaded again; the agents' data holds your sessions and settings. When space runs out, look at old program versions and at models you no longer use first.

Sizes are space on disk: a file kept in two places counts once, and links are not followed.

CHAPTER 11

Healthy & Security check

A coding agent reads and writes files and runs commands on your computer. How much it may do *without asking you* is set in the agent's own settings, with different words in every agent. The **Healthy & Security check** brings this together: on the details page of every agent (**Details** on its card) and of every local LLM, one card shows whether it works, what it may do, what that means in plain words, and what lowers the risk.

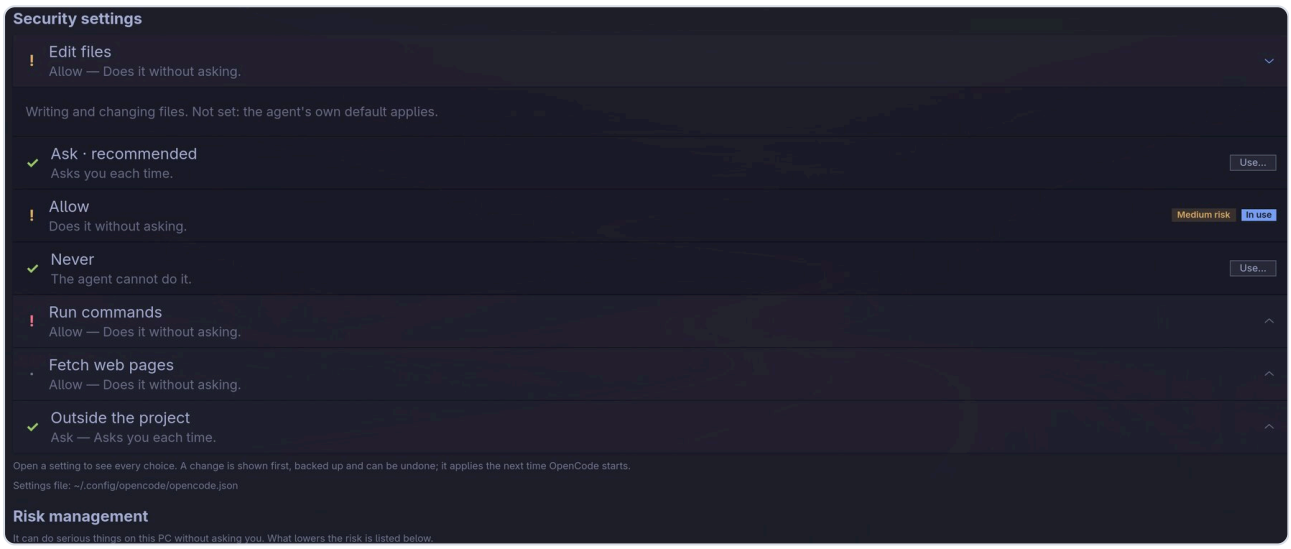


Figure 7 · OpenCode's security settings, with **Run commands** opened, and the start of its risk management.

11.1 Reading the card

Two badges at the top sum it up: **Healthy**, **Needs a look** or **Not set up** for the health, and **Low risk**, **Medium risk** or **High risk** for security. Below them come four parts:

Part	What it shows
Health	Every setup check with a ✓, or what to do next: installed, signed in, its model and local LLM, MCP servers and skills.
Security settings	The agent's settings that decide what it may do on its own, each with its value now and one sentence on what that means. The mark in front tells the risk: ✓ safe, · low, a yellow **!** medium, a red **!** high.
Risk management	What the settings mean together, most serious first, each with why it matters and → what lowers it. Protections that are in place are listed with a ✓.
What the words mean	Sandbox, approval, allow and deny rules and the other words the card uses, explained (see The words explained).

Allow and deny rules and trusted folders the agent has (for example Claude Code's `permissions.allow` list) are listed below the settings, one rule per line.

11.2 Changing a setting

1. Click a setting to open it. You see what it controls and every choice, each with its risk; the one in use is marked **In use**, the safer default **recommended**.
2. Click **Use…** beside a choice, or **Lower it…** beside a risk. A page shows the change in words (old → new), the risk before and after, and **What changes**: the lines of the agent's settings file.
3. Click **Change**. The file is backed up and written; a message offers **Undo** for 30 seconds. The agent uses the new setting the next time it starts.

> **NOTE** Only the one key changes; comments and everything else in the file stay as they are. A setting that needs more than a line in a file — such as Hermes running its commands in Docker — is shown and explained, with where to change it, but not changed from here.

11.3 What is checked for each agent

Agent	Settings shown and changeable	File
Claude Code	Permission mode (Ask first, Plan only, Only what is allowed, Edit without asking, Automatic review, Bypass all checks), sandbox for commands, block bypass mode; allow / ask / deny rules	`~/.claude/settings.json`
Codex	When Codex asks, sandbox (read only, project folder, none), network in the sandbox; trusted folders	`~/.codex/config.toml`
GitHub Copilot	Permission mode (Ask first, Automatic review, Allow everything); trusted folders, sites without asking	`~/.copilot/settings.json`
Cursor CLI	Approval, sandbox, network in the sandbox, web search without asking; allow / deny rules	`~/.config/cursor/cli-config.json`
Hermes	Dangerous commands (always ask, automatic review, never ask), hide secrets; where commands run (shown only)	`~/.hermes/config.yaml`
OpenCode	Ask, allow or never for editing files, running commands, fetching web pages and working outside the project	`~/.config/opencode/opencode.json`
Pi	Its tools: read, write and run commands, or read only. Pi never asks by design, so its tools are its permissions.	`~/.pi/agent/settings.json`

For other agents the card shows their health and their MCP servers, and says where to look in the agent's own settings. For a **local LLM** it shows who can reach it: only this PC (`127.0.0.1` , as it should be) or every device on your network.

11.4 Managing the risk

The risk level is the most serious point the card finds:

- **High risk** — it can do serious things without asking: run any command, or work with no sandbox and no questions. Fine in a throwaway container or virtual machine; on your own PC, let it ask first.
- **Medium risk** — some things happen without asking, such as editing files in the project, or an automatic review decides for you.
- **Low risk** — it asks before anything that matters, or cannot do it at all.

Good habits, whatever the settings:

- **Ask first for commands** on your own computer; allow single, harmless commands (such as `git status`) with an allow rule instead of allowing the whole shell.
- **Switch the sandbox on** where the agent has one (Claude Code, Codex, Cursor CLI), and leave the network in it off unless a task needs it.
- **Start agents in the project folder**, not in your home folder, and keep keys and passwords out of it.
- **Keep MCP servers few**: only the ones you use and trust. Each runs with your rights.
- **Run unattended work** (bypass mode, "YOLO", Pi with all tools) in a container or virtual machine.
- **Local LLMs on** `127.0.0.1`: their servers have no password.

> **CAUTION** An automatic review (Claude Code's *auto* mode, Copilot's *assisted*, Cursor's *auto-review*, Hermes's *smart*) is a second AI judging each step. It saves questions, but it is a judgement, not a wall.

11.5 The words explained

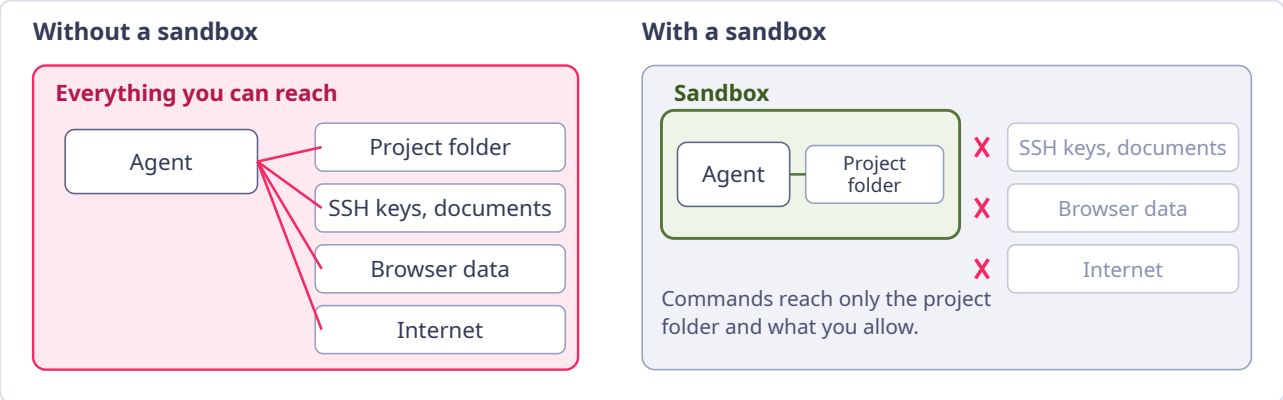


Figure 8 · What a sandbox changes: commands reach only the project folder and what you allow.

Sandbox	A fenced-off area the agent's commands run in. Inside it a command can touch only the project folder (and what you allow) — not the rest of your home folder, your SSH keys or browser data, and often not the internet. Without one, a command the agent runs can do anything you can.
Approval (asking first)	Before the agent edits a file or runs a command, it shows what it wants to do and waits for Yes or No. The less it may do without asking, the more control you keep, and the more often you are asked.
Allow and deny rules	Lists of commands or tools the agent may use without asking (allow) or never (deny). Deny wins over allow. A rule that allows every shell command switches approval off for the shell.
Trusted folder	A folder you said the agent may work in. There it also uses the folder's own instructions and settings, which can come from someone else when you clone a project.
Network access	Whether commands may reach the internet. Code that reads your files and can send data out could leak them; installing packages needs the network too.
Bypass (“YOLO”)	Everything without asking and without limits. Only for a throwaway virtual machine or container.
Automatic review	A second AI model lets harmless actions through and asks about the rest.
Prompt injection	Instructions hidden in a web page, file or tool answer that try to make the agent do something you did not ask for. Asking first, a sandbox and no network stop it.
Secret redaction	Text that looks like a password, token or key is masked before the model sees it or it reaches a log.
Tools	What the agent can do at all: read, write, run commands, search the web. A tool it does not have cannot be misused.
MCP server	A small program that gives the agent extra tools. It runs as you, with your rights.
127.0.0.1 (localhost)	An address only programs on this PC can reach. A server on `0.0.0.0` can be reached from every device on your network.

CHAPTER 12

Privacy and safety

- **No administrator rights.** The app never asks for root.
- **No credentials read.** It reads the agents' settings, but never their credential files. Whether an agent is signed in is judged from whether such a file exists. Values of keys and headers are never shown.
- **No conversations read.** Usage figures are read as numbers and times only.
- **Network only when asked** — searching models, MCP servers or skills, installing, testing a server — plus an update check every few hours.
- **Checked downloads.** Programs are installed only when the download matches the checksum its release publishes.
- **Previews and Undo.** Every change to another program's settings is previewed, backed up and can be undone. Settings that may hold keys are written readable only by you.
- **Bounded.** Every program runs without a shell, and every answer is read with a size and time limit.

The README in each repository lists in full what is read, started and written.

CHAPTER 13

Troubleshooting

Problem	What to do
The map is missing; only cards are shown.	The window is narrower than about 1300 pixels. Maximize it, or press `Ctrl`+`B` to hide the cards.
A box I expect is not on the map.	Check **My setup** vs **+ Available**, the **All · Online · Local LLMs** switch, the **Show** toggles and the column heading's list (a heading like *(5/8)* means some are hidden). Hidden agents are under **+ Available › Hidden**.
Start here is missing or does nothing.	Install VTE for GTK 4 (`vte4`), or use **Start in terminal**.
Agent icons are boxes.	Install a Nerd Font (Symbols Nerd Font).
A line is dotted red.	Click it for hints, fix what it says, then **Test again**. See When a connection fails.
The local LLM does not start with the agent.	Check Settings › **Start local LLMs with their agents**, and that the agent is connected on the map. `systemctl --user status ai-agents-map-providers` shows the watcher.
The coloured light from a local LLM to its model does not show.	It shows only while a program that is not on the map is generating with that LLM, and fades about ten seconds after its last request. Check the map is visible (a wide window) and that the program is actually sending something.
The agent answers badly or cuts off with a local model.	The context may be too small. Open the model under **Models… › Installed** and use **Set context** with the best value for your card.
A download stopped.	It resumes by itself after a broken connection. After a restart, use **Continue** in the strip at the top.
I want to undo a change from earlier.	Backups are kept in `~/.local/state/ai-agents-map/backups/` .

APPENDIX

Reference

A.1 Keyboard shortcuts

`F1`	Help
`Ctrl`+`,`	Settings
`Ctrl`+`B`	Show or hide the cards
`Ctrl`+`+` / `Ctrl`+`-` / `Ctrl`+`0`	Zoom the map in, out, back to 100 %
`Ctrl`+scroll	Zoom the map
`Shift`+click	Add to or remove from the selection
`Esc`	Clear the selection; go back from a page
`Backspace`	Go back from a page
Arrow keys	Move, open and close entries in the tree

A.2 Files and folders

Path	Contents
`~/.config/asterism/` ASTERISM `~/.config/omarchy/ai-agents-map/` OMARCHY	Settings, the MCP server list, looked-up agents, failed connections.
`~/.local/state/ai-agents-map/backups/`	Backups of every file the app changed.
`~/.local/state/ai-agents-map/jobs.json`	Interrupted downloads and installs.
`~/.local/state/ai-agents-map/reorganize.json`	The journal of the last reorganize, for Undo.
`~/.cache/ai-agents-map/`	Scan results, tool lists, update checks.
`~/.config/systemd/user/ai-agents-map-providers.service`	The watcher that starts local LLMs with their agents.

A.3 Glossary

Approval	An agent asking before it changes a file or runs a command (see The words explained).
Context	How many tokens of text a model can keep in mind at once.
GGUF	The file format for models used by llama.cpp, LM Studio, Ollama and KoboldCpp.
Prompt injection	Hidden instructions in a page, file or tool answer that try to steer an agent.
Quantization	Storing a model with fewer bits per weight (for example *Q4_K_M*, about 4.8 bits) so it is smaller, with a small loss of quality.
Mixture of experts	A model where only part of the parameters work for each token (*35B-A3B*: 35 billion in all, 3 billion active), which makes it fast for its size.
Sandbox	A fenced-off area an agent's commands run in, reaching only the project folder and what you allow.
Token	A piece of text, roughly three quarters of a word, that models read and write.
User service	A background program started by systemd for your user only, without administrator rights.